Ex No: 1

IMPLEMENT EIGHT QUEENS PROBLEM

Aim:

To Implement Eight Queens Problem using backtracking

Procedure:

Program

```
N = 8  # Size of the chessboard (8x8)

# Function to print the solution
def printSolution(board):
    for row in board:
        for i in range(N):
            print("Q" if row[i] else ".", end=" ")
        print()
    print()  # Add a newline for readability

# Function to check if a queen can be placed on board[row][col]
def isSafe(board, row, col):
    # Check the column
    for i in range(row):
        if board[i][col]:
            return False
```

```
    # Check the upper left diagonal
    for i, j in zip(range(row - 1, -1, -1), range(col - 1, -1, -1)):
        if board[i][j]:
            return False

    # Check the upper right diagonal
    for i, j in zip(range(row - 1, -1, -1), range(col + 1, N)):
        if board[i][j]:
            return False
    return True

# Function to solve the 8 Queens problem using backtracking
def solve(board, row):
    if row == N:
        printSolution(board)
        return True  # Stop after finding one solution

    for col in range(N):
        if isSafe(board, row, col):
            board[row][col] = 1  # Place queen

            if solve(board, row + 1):  # Recur to place next queen
                return True  # Stop after first valid solution

            board[row][col] = 0  # Backtrack (remove queen)

    return False

# Main function to initialize the board and start solving the problem
def eightQueens():
    board = [[0 for _ in range(N)] for _ in range(N)]
```

```
    if solve(board, 0):
        print("One possible solution is:")
    else:
        print("Solution does not exist")

# Calling the function
eightQueens()
```

Output:

```
One of the possible solutions is:
. . . . Q . . .
. . . Q . . . .
. . . . . Q . .
Q . . . . . . .
. . . . . . Q .
. Q . . . . . .
. . Q . . . . .
. . . . . . . Q
```

```
========== RESTART: C:/Users/ASUS/AppData/Local/Programs/Python/Python313/ghjkl
Q . . . . . .
. . . Q . . .
. . . . . . Q
. . . . . Q .
. . Q . . . .
. . . . . Q .
. Q . . . . .
. . . Q . . . .

One possible solution is:
>>> 0
0
```